Test Plan: Amazon Pay Balance Functionality

Project Name: Amazon Pay Balance (Wallet and Gift Card)
Version: 1.0
Date: 11-02-2026
Platform: amazon.in web

Disclaimer:

- All the Section mentioned below are mandatory and should not be deleted.
- In case if any section is not applicable for your project, mark it as NA
- This is a standard template which cover important and mandatory section of the plan, please do not restrict yourself only with these and based on Requirement feel free to add more section or sub section in your Test Plan.

Reference Document

Version	Document Type	Document Name/Url
0.1	BRD (Business Requirement Document)	Observed live
0.1	UI Mockup /Design document	Observed
0.1	Project wiki Link	N/A

Final Design Review and Approval

Name	Role	Approver/Reviewer	Approval / Review Date / Comments
	Ux Designer	Approver	
	SDM	Approver	
	PM	Approver	
	QA	Reviewer	
	Dev	Approver	

GEN AI CONSUMPTION AND SAVE [Overall Save ~ X days/hours]

Phase	Usage	Impact [Save in Days]	Tool Used
Requirement Analysis Phase			N/A
Test Planning Phase	Test plan structure, scenario identification	1 Day	ChatGPT
Test Execution Phase	Test case generation, edge case identification	0.5 Day	ChatGPT
Automation			
Report Generation			

1. Introduction

Amazon Pay balance can be added to an account in two ways: through the wallet or via gift card redemption. Wallet money can be added by the user themselves or by a known person with access to the account, and it can be used for Amazon shopping, merchant payments, or sent to friends/family only if the user's bank account is linked; otherwise, transfers are not allowed. Gift card balance, received as a gift, can be redeemed using the gift card code, after which it is added to the wallet and can be used for shopping or merchant payments, but cannot be sent to friends or family members. Once money is added via wallet or gift card, the Amazon Pay balance shows the amounts separately (e.g., Wallet: ₹200, Gift Card: ₹4000) along with the total balance (Amazon Pay Balance: ₹4200).

1.1 Acceptance Criteria

- The user should be able to add money to the Amazon Pay wallet after successful login.
- The user should be able to redeem a gift card using a valid gift card code.
- The user should be able to use the available wallet balance (including redeemed gift card balance) for: Amazon shopping, Merchant payments
- The user should not be able to transfer or send gift card balance to friends or family members. Gift card balance must be restricted to shopping and merchant payments only.
- The user should be able to send money to friends or family members using wallet balance, subject to applicable conditions (e.g., bank account linking, if supported).
- If a guest user attempts to add money to the wallet without logging in, the system should redirect the user to the login page.
- Only authenticated (logged-in) users should be able to add money to the wallet.
- If another person has valid access to the user's Amazon account and is logged in, they should be able to add money to the wallet.
- When money is added through wallet top-up, the amount should be stored under the Wallet Balance section and gift card is redeemed, the amount should be stored under the Gift Card Balance section.
- When a purchase is made:
    - If the amount is deducted from Gift Card balance, it should reduce only the Gift Card balance and update the Amazon Pay Balance (total) accordingly.
    - If the amount is deducted from Wallet balance, it should reduce only the Wallet balance and update the Amazon Pay Balance (total) accordingly.
- After every transaction (top-up, redemption, payment), the individual balances and total balance must be updated accurately and stored correctly in the system.
- If the user has insufficient wallet or gift card balance while making a purchase, the system should prompt the user to add money to the wallet and allow the user to complete the payment after successful top-up

2. Scope

- Add money to wallet
- Redeem gift card
- Balance display (Wallet, Gift Card, Amazon Pay Balance)
- Deduction logic during shopping
- Transfer wallet money (if supported)
- Login validation before wallet access
- Error handling for invalid gift card

2.1 In Scope

Form Factor Coverage –

Form Factor	Applicable?
Android	☑
iOS	☐
mWeb	☐
Desktop	☑

2.2 Out of Scope

- Bank-side transaction failures
- Refund Process
- Transaction History
- Add Cash to Balance (Cash on Delivery / offline flow)

3. Test Strategy

3.1 Test Strategy Checklist

Test Type	Applicable
Functional Testing	☑
Monkey Testing	☑
Performance Testing	☑
Localization Testing	☑
Regression Testing	☑
Compatibility Testing	☑

3.1.1 Test Strategy Detail

- **Functional Testing:** Functional testing of the Amazon Pay balance includes not only positive test scenarios but also negative testing, boundary value testing, and edge case validation for both Wallet and Gift Card features. This ensures that valid transactions such as adding money to the wallet, redeeming gift cards, and making payments work correctly, while invalid scenarios like incorrect gift card codes, expired vouchers, insufficient balance, and unauthorized access are handled gracefully with proper error messages. Boundary and edge cases such as minimum and maximum add-money limits, zero balance conditions, partial payments, and transaction failures are also verified to ensure system stability, data accuracy, and a reliable user experience across all wallet and gift card operations.

- **Monkey Testing:** Monkey testing will be performed to validate the stability of the Amazon Pay wallet and gift card UI by executing random user actions such as clicking buttons, navigating between pages, entering random inputs, and interrupting payment flows to ensure the application does not crash and handles unexpected behaviour gracefully.

- **Performance testing:** Performance testing will be conducted for the Amazon Pay balance features to evaluate system responsiveness, stability, and scalability under normal and peak load conditions. This includes measuring response time and latency for critical user journeys such as adding money to the wallet, redeeming gift cards, viewing balance breakdown (Wallet vs Gift Card), and completing merchant payments. Load and stress scenarios will be executed to ensure the system can handle concurrent users and high transaction volumes without degradation, while monitoring failures, timeouts, and graceful error handling during network or payment gateway delays.

- **Localization Testing:** Localization testing will be performed for the Amazon Pay features to ensure the application correctly supports different languages, regions, currencies, date/time formats, and cultural conventions. This includes validating translated UI text for Wallet and Gift Card flows, correct currency symbols and formatting for balances and transactions, proper display of numbers and decimal separators, and region-specific content such as payment options and error messages. The goal is to ensure users across supported locales have a consistent, accurate, and user-friendly experience without functional or UI issues.

- **Regression Testing:** Regression testing will be performed for the Amazon Pay balance features to ensure that new changes, bug fixes, or enhancements do not break existing functionalities such as adding money to the wallet, redeeming gift cards, balance calculation (Wallet vs Gift Card vs Amazon Pay Balance (Total)), and merchant payments. Critical user flows and previously fixed defects will be re-validated across web and mobile platforms. A regression suite will be maintained and executed for every release, with stable scenarios automated where feasible and complex payment or external dependency flows validated manually to ensure end-to-end stability.

- **Compatibility Testing:** Compatibility testing will be conducted for the Amazon Pay features to ensure consistent behaviour and UI across different platforms, devices, browsers, operating systems, and screen resolutions. This includes validating Wallet and Gift Card flows on major web browsers (Chrome, Firefox, Edge) and on mobile devices across supported Android and iOS versions. The application should function correctly with different network conditions and device configurations, ensuring a seamless and reliable user experience for all supported environments.

3.1.2 TEST COVERAGE

OS Coverage

	A	B
1	**Priority**	**Operating System**
2	**P0**	Windows
3	**P1**	Android 15

3.2 Automation Plan

UI Automation Strategy / Approach

- Automate critical user journeys
- Focus on smoke and sanity tests
- Page Object Model (POM) design pattern
- Cross-platform execution (Web & Android)

Packages/Libraries

- Selenium WebDriver 4.23.0
- TestNG 7.11.0

Functionality	Priority	E2E Coverage
Amazon Pay -> Add Money -> Enter Amount -> Add money to Wallet -> Select Payment Method -> payment ->Successful Payment (Happy path)	P0	Yes
Amazon Pay -> Enter Gift Card code -> Add gift card to balance (Happy path)	P0	Yes
Payment failure flow	P0	Yes
Smoke test suite (critical scenarios)	P0	Yes

Note: E2E testing covers complete user flows from bill selection to payment confirmation. Partial coverage means individual features tested but not full E2E.

3.3 Test Environments

Environments	Purpose	User Accounts	Data Type	Access
Dev	Development and Dev Testing	Mock only	Synthetic (Fake)	VPN
QA/Beta	Maintain Testing Environment	Mock + limited real accounts	Test Data	VPN
Pre-Prod/Gamma	UAT & final Validation	Limited real accounts	Sanitized Prod Data	VPN
Production	Live Environment	All Live Accounts	Live Data	Public

3.4 Test Data

1. Test Data – Add Money Amount Field

- **Valid Data:** 1, 786, 10000
- **Invalid Numeric Data:** -1, -788, 0
- **Boundary Values:** Max limit (e.g:100000), Above max (e.g:100001), Min limit (e.g:1)
- **Special Characters:** %$%$*&^
- **Alphabetic Input:** ABHG

2. Test Data – Add Money Amount Field

- **Valid Data:** WU56WTT9KVCGH5
- **Invalid Data:** jhvgyfghbgfh
- **Alphabetic Data:** FDERRKHYUTURJGB
- **Numeric Data:** 76978745327455
- **Empty Input:** (blank)
- **Special Characters:** ^&^%%^%T(
- **Expired Code:** (use an expired test gift card code)

Account Type	UserName	Purpose
Regular	testuser1@example.com	Verify login, add money, redeem gift card, and complete shopping/payment.
Prime	primeuser@example.com	Verify login, add money, redeem gift card, and complete shopping/payment.
Guest	--	Verify redirection to login page when accessing Amazon Pay features

3.5 Test Scenarios

No	Module	Use Case	Remark
1	Amazon Pay	Validate navigation to Amazon Pay page from Home page	Functional, Positive Test
2	Amazon Pay	Validate Add Money and Gift Card options under Amazon Pay balance section	Functional, Positive Test
3	Amazon Pay	Validate navigation to Add Money page from Amazon Pay home page	Functional, Positive Test
4	Amazon Pay	Validate navigation to Gift Card page from Amazon Pay home page	Functional, Positive Test
5	Amazon Pay	Validate adding money to Wallet	Functional, Positive Test
6	Amazon Pay	Validate redeeming Gift Card to balance	Functional, Positive Test
7	Amazon Pay	Validate total balance calculation and display	Functional, Positive Test
8	Amazon Pay	Validate Wallet and Gift Card balances under Total Balance	Functional, Positive Test
9	Amazon Pay	Validate navigation to Add Gift Card page from "Do more with Amazon Pay Balance" section	Functional, Positive Test
10	Amazon Pay	Validate navigation to Amazon Pay Balance page from "Add Money" section	Functional, Positive Test
11	Amazon Pay	Validate Amazon Pay page content updates correctly after changing the site language on Amazon.in	Functional, Localization Test

3.6 Test Cases

- **GIFT CARD & WALLET – 300 WEB UI TEST CASES**

- Verify Gift Card page loads successfully
- Verify correct page title displayed
- Verify correct URL loaded
- Verify Gift Card section visible
- Verify Gift Card input field present
- Verify input field enabled
- Verify placeholder text visible
- Verify cursor appears on click
- Verify user can enter text
- Verify alphanumeric input accepted
- Verify special characters rejected
- Verify max length validation
- Verify min length validation
- Verify leading spaces trimmed
- Verify trailing spaces trimmed
- Verify multiple spaces handled
- Verify copy functionality
- Verify paste functionality
- Verify invalid paste handled
- Verify field clears on refresh
- Verify Redeem button visible
- Verify Redeem button enabled for valid input
- Verify Redeem disabled when empty
- Verify Redeem disabled for invalid input
- Verify button click triggers action
- Verify hover state
- Verify active state
- Verify disabled state
- Verify multiple clicks prevented
- Verify loader appears
- Verify loader disappears
- Verify success message displayed
- Verify success message color
- Verify success message alignment
- Verify success message content
- Verify success message timeout
- Verify error message displayed
- Verify error message color
- Verify error message alignment
- Verify error message content
- Verify invalid code error
- Verify expired code error
- Verify used code error
- Verify empty field error
- Verify error disappears after correction
- Verify no overlapping messages
- Verify balance section visible
- Verify Gift Card balance shown
- Verify total balance shown
- Verify balance updates
- Verify balance format
- Verify balance persists after refresh
- Verify balance persists after login
- Verify redemption reflected in UI
- Verify no duplicate redemption UI
- Verify login required for redemption
- Verify session expiry redirects to login
- Verify return to same page after login

- Verify back navigation safe
- Verify refresh safe
- Verify Wallet section visible
- Verify Wallet balance visible
- Verify Wallet format correct
- Verify Wallet updates after redemption
- Verify Wallet updates after add money
- Verify Wallet persists after refresh
- Verify Wallet persists after login
- Verify Wallet visible globally
- Verify Wallet icon clickable
- Verify navigation to Wallet
- Verify Add Money button visible
- Verify Add Money clickable
- Verify Add Money popup opens
- Verify popup layout correct
- Verify amount input visible
- Verify numeric input only
- Verify reject alphabets
- Verify reject special characters
- Verify min amount validation
- Verify max amount validation
- Verify invalid amount error
- Verify Continue enabled valid input
- Verify Continue disabled invalid input
- Verify Continue navigates next
- Verify Cancel closes popup
- Verify close icon works
- Verify payment options visible
- Verify payment option selectable
- Verify selection highlight
- Verify proceed button visible
- Verify proceed enabled
- Verify success UI
- Verify failure UI
- Verify retry option
- Verify Wallet updates on success
- Verify Wallet unchanged on failure
- Verify transaction history visible
- Verify navigation to history
- Verify list displayed
- Verify latest first
- Verify transaction amount
- Verify transaction type
- Verify transaction date
- Verify scroll works
- Verify pagination
- Verify empty state
- Verify filter visible
- Verify filter works
- Verify filter results
- Verify clear filter
- Verify Gift Card purchase page loads
- Verify purchase layout
- Verify predefined amounts
- Verify custom amount field
- Verify custom amount validation
- Verify recipient name field
- Verify recipient email field
- Verify recipient phone field
- Verify message field
- Verify field validations

- Verify schedule delivery option
- Verify date picker
- Verify valid date selection
- Verify invalid date blocked
- Verify delivery summary
- Verify buy button visible
- Verify buy button enabled
- Verify buy button click
- Verify confirmation screen
- Verify purchase success UI
- Verify purchase failure UI
- Verify Gift Card appears after purchase
- Verify Gift Card usable after purchase
- Verify checkout page loads
- Verify Gift Card auto applied
- Verify Wallet auto applied
- Verify toggle Wallet usage
- Verify updated total
- Verify insufficient balance message
- Verify payment summary
- Verify order success UI
- Verify order failure UI
- Verify retry payment UI
- Verify Wallet deducted
- Verify Wallet not deducted on failure
- Verify Gift Card deducted
- Verify refund message
- Verify Wallet refund update
- Verify Gift Card refund update
- Verify refund in history
- Verify UI responsive desktop
- Verify UI responsive tablet
- Verify UI responsive mobile
- Verify layout consistency
- Verify alignment consistency
- Verify spacing consistency
- Verify font consistency
- Verify button alignment
- Verify input alignment
- Verify icon alignment
- Verify hover effects
- Verify disabled elements not clickable
- Verify spinner shown
- Verify spinner removed
- Verify UI not frozen
- Verify refresh keeps state
- Verify back navigation
- Verify forward navigation
- Verify multi-tab behavior
- Verify session consistency
- Verify logout visible
- Verify logout works
- Verify redirect after logout
- Verify login again UI
- Verify login error UI
- Verify login success UI
- Verify session timeout UI
- Verify re-login flow
- Verify currency symbol display
- Verify decimal precision
- Verify number formatting
- Verify long values formatting

- Verify zero balance display
- Verify empty field handling
- Verify large input handling
- Verify whitespace input
- Verify unicode input
- Verify emoji input
- Verify error for invalid format
- Verify multiple errors handling
- Verify inline validation
- Verify validation on blur
- Verify validation on submit
- Verify validation message removal
- Verify field highlight on error
- Verify field normal after correction
- Verify UI consistency across pages
- Verify consistent colors
- Verify consistent fonts
- Verify consistent buttons
- Verify focus moves correctly
- Verify tab navigation
- Verify keyboard input
- Verify enter key submits
- Verify escape closes popup
- Verify accessibility labels
- Verify ARIA attributes
- Verify screen reader text
- Verify image/icons load
- Verify broken image handling
- Verify fallback UI
- Verify error page UI
- Verify retry UI
- Verify timeout UI
- Verify Gift Card list UI
- Verify multiple Gift Cards display
- Verify Gift Card selection
- Verify Gift Card details UI
- Verify Gift Card expiry shown
- Verify Gift Card status shown
- Verify Wallet transaction types UI
- Verify credit UI
- Verify debit UI
- Verify pending UI
- Verify failed UI
- Verify button spacing
- Verify input spacing
- Verify section spacing
- Verify responsive breakpoints
- Verify scroll behavior
- Verify modal open animation
- Verify modal close animation
- Verify overlay background
- Verify modal focus trap
- Verify link navigation
- Verify breadcrumb UI
- Verify home navigation
- Verify back to dashboard
- Verify theme consistency
- Verify light mode
- Verify dark mode
- Verify language switch UI
- Verify translated labels
- Verify translated errors

- Verify layout after language change
- Verify tooltip display
- Verify tooltip content
- Verify tooltip position
- Verify button loading state
- Verify button disabled state UI
- Verify session expiry message
- Verify re-authentication UI
- Verify warning messages
- Verify confirmation dialogs
- Verify cancel action UI
- Verify Gift Card cannot be edited
- Verify Wallet cannot be manually edited
- Verify consistent UI after multiple actions
- Verify repeated navigation consistency
- Verify correct icon usage
- Verify icon click actions
- Verify no overlapping UI elements
- Verify no clipping issues
- Verify scroll to top behavior
- Verify sticky header/footer
- Verify error page navigation
- Verify fallback UI consistency
- Verify UI after network reconnect
- Verify retry UI after reconnect
- Verify disabled button styling
- Verify multi-language alignment
- Verify RTL layout (if applicable)
- Verify empty Gift Card UI
- Verify empty Wallet UI
- Verify data refresh UI
- Verify manual refresh action
- Verify UI stability on rapid clicks
- Verify UI stability on repeated actions
- Verify no duplicate UI rendering
- Verify clean UI transitions
- Verify logout from multiple tabs UI
- Verify UI sync across tabs
- Verify UI after browser resize
- Verify UI after zoom in/out
- Verify UI after clearing cache
- Verify UI after cookie reset
- Verify UI consistency after updates
- Verify UI reload behavior
- Verify end-to-end UI flow Gift Card → Wallet
- Verify Wallet → Checkout → Refund UI
- Verify full lifecycle UI consistency
- Verify UI after multiple transactions
- Verify final balance display accuracy
- Verify UI correctness after all flows
- Verify user journey smoothness
- Verify UI responsiveness
- Verify UI error recovery
- Verify UI fallback handling
- Verify overall UI usability
- Verify complete end-to-end UI experience

3.7 Defects

Defect ID:
Module Test Case ID:
Title / Summary Description:
Steps to Reproduce:
Test Data:
Expected Result:
Actual Result:
Severity:
Priority:
Environment:
Status:
Assigned To:
Attachments:

4. Assumptions and Dependencies

Assumptions:

- Test environment is available and stable
- Test user accounts (regular/prime) are pre-created
- Valid and invalid gift card codes are available
- Requirements and UI are frozen before test execution

Dependencies:

- Build deployment from Dev team
- Backend APIs and payment services availability
- Test data setup from DB/Dev team
- VPN access for QA environment

5. Risks and Contingencies

Risks:

- Unstable QA environment may block testing
- Delayed build delivery from Dev team
- External payment service downtime
- Last-minute requirement changes
- Unstable build from the Dev team

Contingencies:

- Use mock services or alternate environment
- Execute smoke tests first when build is delayed
- Test core flows using stubs if payment service is down
- Prioritize P0 scenarios if scope changes

6 Test Deliverables

S.No.	Deliverable Name	Author	Reviewer
1	Test Strategy and Test Plan	QAE@	QA Team, QAM,Dev Team/SDM
2	Test Cases	QAE@	QA Team/Dev Team/PM
3	Test Data	QAE@/PM@	QAM/SDE/SDM/PM/TPM
4	API Testing & IN-Sprint planned automation	QAE@	
5	First Test pass	QAE@	
6	Daily status report (including bug reports)	QAE@	QAM/SDM/TPM/PM/SDE/Clients QA if any
7	Defect Fix by Dev	SDE@	
8	Production Sanity	QAE@	
9	Defect Validation	QAE@	
10	Sanity & Sign-Off	QAE@	QAM/SDM/TPM/PM/SDE
11	Create sanity cases if required and update on the Intake	QAE@	
12	Share Quality metric	QAE@	QAM/SDM/TPM/PM/SDE
13	Client QA Sign Off		

7 Dev Build Plan

Build Version	ETA	Notes

8 QA Acceptance/Entry Criteria

Criteria	Status	Comment and ETA
Developer to Execute BVT Cases before build drop BVT List - here		
Developer to document and share all Code Review feedback in Quip, with descriptions of the associated functional changes and coverage		
All the Open Questions are addressed		

Weblab detail with access in place		

9 QA Exit Criteria

Task	Status	Comment and ETA
100 % Test cases Executed.		
100% Test Strategy is covered and completed		
All open issue are triaged and got alignment to differ.		
Planned Automation completed		
All the Open Bugs are triaged and have Product alignment		
Client Sign off are in place (If Applicable)		
Test Rail is updated		

10 Open Questions

Mention all the Open questions present during the design/development/review & execution phases.

- Parent sim containing all the issues
- Track any outstanding issues, owner for follow up and status. This list typically represents QA questions that are not answered yet.

11 Effort Estimation / person

Task Name	Ownership	Start Time	Finish Time	Status	Comments(If any)
Requirement Understanding					
Test Planning					
Test cases writing					
Test plan Review					
Test Data collection if required					
Planned buffer					
First Pass completion					
Bug creation, Retest the bug and Triage					
Retest the failed cases and Sanity					
Any test planned to test in prod					

QA LEAD SIGN-OFF

Prepared By:
Name: QA Lead
Date: February 25, 2026
Signature: _____

Reviewed By:
QA Manager: _____
Development Manager: _____
Product Manager: _____

Final Approval:
SDM: _____
Date: _____